Compiler Construction

Translations

Tree Compiler Overview

What ?

typed AST (TAST) $\rightarrow$ High-Level Intermediate Representation (HIR).

- Input: typed abstract syntax tree
- Output: textual HIR code
- Strategy: syntax-directed, visitor-based compilation

Each AST node emits HIR code *immediately*.

Architecture: Visitor-Based Compilation

- One visitor method per AST node
- Each method emits HIR code
- Recursive structure follows the AST

Key property

Compilation is **local**:

- No global rewriting pass
- No delayed decisions
- No backpatching phase

Code Emission Infrastructure

- Output is produced incrementally
- Indentation mirrors HIR tree structure

Mechanism

- `emit(String)` prints text
- `indent()` / `dedent()` control layout

This makes the HIR both:

- structured
- readable

ILP-1

ILP1's code emission.

Litterals

- `42` → `const 42`
- `42.0` → `constF 42.0`

No dedicated boolean type in HIR

- `true` → `const 1`
- `false` → `const 0`

Consequences

Conditions are integer comparisons against 0

Temporaries

Purpose

Temporaries store intermediate computation results.

- Integers / booleans → `t0`, `t1`, ...
- Floating-point values → `f0`, `f1`, ...

Properties

- Fresh temp per binding or expression result
- Never reused
- Explicitly named in HIR

Moves

Assignments

- `move temp t e2`
  Evaluate e2 and move it into temporary t.
- `move e1 e2`
  Evaluate e1, yielding address a. Then evaluate e2, and store the result into memory starting at a.

Our IR is not scoped ! Every temporary is defined **globally**

Exercise - 1

Translate to hir

```
x = 1 + 41
```

```
x = 1.5 + 40.5
```

we'll ignore the fact that ILP's assignment return the value of the initializer

Variable Environment

Representation

A stack of scopes:

```
Deque<Map<String, String>>
```

- Each scope maps variable names ⟶ temp names
- Stack supports lexical scoping
- Inner scopes shadow outer bindings

Variable lookup searches from innermost scope outward.

Example: Variable Binding

```
let x = 3 in x + 1
```

- Allocate a fresh temp for `x`
- Emit a `move` instruction
- Record $\texttt{x} \mapsto \texttt{t0}$ in the environment

All future occurrences of `x` emit `temp t0`.

Exercise - 2

Translate to hir

```
let x = 40
and y = 2 in
x + y
```

```
let x = 3 in
(let x = 37
 and y = x+2
 in x + y)
```

String Handling

Assembly (and Tree) have string declared in a a dedicated section.

Problem

String literals may appear anywhere in the program.

Solution

A preliminary **string collection pass**.

- Traverses the AST
- Collects all string literals
- Assigns each a unique label

String Emission

At program start

```
label L0 "hello"
label L1 "world"
```

At use site

```
name L0
```

Strings are treated as global, immutable values.

Function Calls

Easy approach:

- Each argument emits its own code
- No dedicated argument registers in the tree IR

Depending on the return value:

- `call` for integer/bool/string
- `callF` for floating-point

Lower-level calling conventions are deferred.

HIR Runtime

String:

- `string: concat`
- `int: strcmp (a: string, b: string)` Compare the strings a and b. Return -1 if a < b, 0 if equal, and 1 otherwise.

Printing

- `void: print`
- `void: print_int`

Standard conversions `string_of_int, float_of_int ...`

Exercise - 3

Translate to hir

```
2.0 + 40
```

```
let x = "hello " in
x + "world"
```

Labels

Purpose

Labels identify control-flow targets.

- Generated via a global counter
- Always fresh and unique
- Used for:
  - conditionals
  - short-circuit logic
  - sequencing

Using Booleans in Control Flow

Conditional evaluation

Conditions are evaluated as integer expressions.

- 0 means *false*
- any non-zero value means *true*

Implementation pattern

```
cjump ne <condition> const 0
```

Same pattern is used for `if`, logical operators, and conditional expressions.

Conditionals as Expressions

Key design choice

`if` is an *expression*, not a statement.

- Condition evaluated
- One branch executed
- Result stored in a temporary

This is implemented using:

- `cjump`
- labels
- `move` into a result temp

Sequencing and Side Effects

Problem

Expressions may have side effects.

HIR solution

`eseq` and `seq`

- `sxp e` → evaluate expression for its effects
- `eseq s e` → run statements, then return value

Bridging Disparities

Tree's `print` returns nothing Tree's `sequences` returns nothing

Prints are

- Compiled as a side-effect-only call `(sxp)`
- Wrapped in an `eseq` returning false (`const 0`)

Sequences

- are compiled as `eseq`
- return the last instruction

Ensures uniform treatment of expressions.

Exercise - 4

Translate to hir

```
(print("hello ");
print("world\n"))
```

```
if true then 1 else 2
```

Summary

- Syntax-directed, visitor-based
- Every expression produces a value
- Explicit control flow via labels
- Explicit storage via temporaries
- No global optimization or rewriting

Result

A simple, predictable, and explicit HIR generator.

ILP-2

ILP2's code emission.

ILP2: Compiler Extension Overview

What ILP2 adds

- Function definitions
- Function calls
- Assignments
- Loops

- Extends the ILP1 tree compiler
- Reuses temporaries, labels, environments
- Introduces a notion of *routines*

Function Name Resolution

Problem

Functions may be mutually recursive.

Solution

Assign a label to each function *before* emitting code.

- Function name $\longrightarrow$ label mapping
- Stored in a global map
- Used during call compilation

Function Labels

Before code generation:

- f $\rightarrow$ L1
- g $\rightarrow$ L2

During code generation

```
label L1    /* function f */
...
label L2    /* function g */
```

One special label `label end` to end each routine (and return to the caller)

This enables direct jumps to function entry points.

Function Parameters

Calling convention (tree-level)

- Integer arguments: `i0, i1, ...`
- Floating-point arguments: `fi0, fi1, ...`

- Parameters are bound at function entry
- Stored directly in the environment
- No stack or registers at this level

Function Body and Return Value

Return convention

- Integer / boolean / string → `rv`
- Floating-point → `fv`

- Function body computes a value
- Value is moved into return temp
- Control falls through to `label end`

Function Invocation

Two cases

- Primitive functions → ILP1 compilation
- User-defined functions → fill environment and call

- Call instruction emits argument values
- Arguments are evaluated in any order
  Tree and ILP implement the same evaluation order
- Return value is an expression result

Assignment Translation

Ensures that assignments yield a value

```
eseq
  move
    temp t0
    <expression>
  temp t0
```

Sequences of assignments `x=42; y=43` will be compiled to sequences of `sxp` statements:

```
seq
  sxp eseq move t22 const 42 t22
  sxp eseq move t23 const 43 t23
seq end
```

Loop Generation Schemes

Observation

A loop can be translated in several equivalent ways in HIR.

- Test before the body
- Test after the body
- Infinite loop with explicit exit

All schemes use:

- labels
- conditional jumps
- backward jumps

Exercise - 5

Describe how to translate loops using labels and jumps.

1. **Test-first loop** (`while`)
   - Condition evaluated before the body
   - Body may execute zero or more times

2. **Body-first loop** (`do--while`)
   - Body executed before the condition
   - Body executes at least once

3. **Infinite loop with explicit exit**
   - Loop has no implicit termination
   - Exit controlled by a conditional jump

Scheme 1: Test-First Loop

Structure

Condition evaluated before each iteration.

```
label Ltest
cjump ne <cond> const 0 Lbody Lend
label Lbody
  sxp <body>
  jump Ltest
label Lend
```

The body may execute zero or more times.

Scheme 2: Body-First Loop

Structure

Body executed before testing the condition.

```
label Lbody
  sxp <body>
  cjump ne <cond> const 0 Lbody Lend
label Lend
```

The body executes at least once.

Scheme 3: Infinite Loop with Exit

Structure

Loop runs indefinitely until a condition triggers an exit.

```
label Lloop
  cjump ne <cond> const 0 Lbody Lend
label Lbody
  sxp <body>
  jump Lloop
label Lend
```

Often used as a basis for translating `break`.

Comparing Loop Schemes

Scheme	Test position	Min iterations	Labels
Test-first	Before body	0	3
Body-first	After body	1	2
Infinite + exit	Inside loop	0	3

All schemes are representable at the HIR level.

- Some schemes have a single entry point
- Some isolate the loop test in one place
- Some make loop exits explicit and uniform

In later sessions this will guide which loop schemes are preferred in practice.

Exercise - 6

Given α, β two integer ILP expressions,

How to translate the expression
$\alpha < \beta$ from ILP into HIR?

Naive translation

With α' and β' the HIR translations of α and β:

```
eseq
    seq
        cjump lt α' β' name ltrue name lfalse
        label ltrue
        move temp t const 1
        jump name lend
        label lfalse
        move temp t const 0
        label lend
    seq end
    temp t
```

Closer look to the naive translation

Naive translation is costly

- one cjump
- one jump
- two label
- one temporary

$\Rightarrow$ Can we do better?

Note: *jumps and cjumps are costly (relatively) with modern microprocessor*
$\Rightarrow$ We must try to minimize them!

Can we leverage additional contextual information?

```
let
    ...
in
   α < β,
   ...
end
```

In this situation we don't care about the translation of $<$
$\Rightarrow$ We are only interested about side effect of α and β

Improved translation

```
seq
  sxp α′
  sxp β′
seq end
```

Improved translation

```
seq
  sxp α′
  sxp β′
seq end
```

- 0 cjump / 0 jumps
- 0 label
- 0 temporary

$\Rightarrow$ Way better! But very circumstantial.

Yet another example

```
let
    ...
in
    if α < β then
        /*TRUE*/
    else
        /*FALSE*/
end
```

In this situation a naive translation would produce a lot of useless jump/cjump

Improved translation

```
cjump lt α′ β′ name ltrue name lfalse
label ltrue
/* TRUE translation */
jump name lend
label lfalse
/* FALSE translation */
label lend
```

Only one cjump (and one jump an the end of ltrue)!
$\Rightarrow$ Better than the naive translation!

Translating Conditions

What is the right translation for $\alpha < \beta$, with α and β two arbitrary expressions?

It depends on how the expression is *used*:

1. `if` $\alpha < \beta$ `then ...`
2. `a :=` $\alpha < \beta$
3. $(\alpha < \beta,\ ())$.

Problem statement

When the visitor is about to translate $\alpha < \beta$, it does not know the context.

Context Sensitive Translation

- The right translation depends upon the *use*. **This is context sensitive!**
- How to implement this?
  - When entering an *IfExp*, warn “I want a condition”
  - then, depending whether it is an expression or a statement, warn “I want an expression” or “I want a statement”.
- Don’t forget to preserve the demands of higher levels...
- That’s a whole new stack that stores context, Eek.

Prototranslation, Expression Shells

Rather, delay the translation until the use is known:

- **Ex** Expression shell, encapsulation of a proto value,
- **Nx** Statement shell, encapsulating a wannabe statement,
- **Cx** Condition shell, encapsulating a wannabe condition.

Then, ask them to finish their translation according to the use:

Exp	un_nx	un_ex	un_cx (t, f)
Ex(e)			
Cx(a < b)			
Nx(s)			

Prototranslation, Expression Shells

Rather, delay the translation until the use is known:

- **Ex** Expression shell, encapsulation of a proto value,
- **Nx** Statement shell, encapsulating a wannabe statement,
- **Cx** Condition shell, encapsulating a wannabe condition.

Then, ask them to finish their translation according to the use:

Exp	un_nx	un_ex	un_cx (t, f)
Ex(e)	sxp(e)		
Cx(a < b)			
Nx(s)			

Prototranslation, Expression Shells

Rather, delay the translation until the use is known:

- **Ex** Expression shell, encapsulation of a proto value,
- **Nx** Statement shell, encapsulating a wannabe statement,
- **Cx** Condition shell, encapsulating a wannabe condition.

Then, ask them to finish their translation according to the use:

Exp	un_nx	un_ex	un_cx (t, f)
Ex(e)	sxp(e)	e	
Cx(a < b)			
Nx(s)			

Prototranslation, Expression Shells

Rather, delay the translation until the use is known:

- **Ex** Expression shell, encapsulation of a proto value,
- **Nx** Statement shell, encapsulating a wannabe statement,
- **Cx** Condition shell, encapsulating a wannabe condition.

Then, ask them to finish their translation according to the use:

Exp	un_nx	un_ex	un_cx (t, f)
Ex(e)	sxp(e)	e	cjump(e $\neq$ 0, t, f)
Cx(a $<$ b)			
Nx(s)			

Prototranslation, Expression Shells

Rather, delay the translation until the use is known:

- **Ex** Expression shell, encapsulation of a proto value,
- **Nx** Statement shell, encapsulating a wannabe statement,
- **Cx** Condition shell, encapsulating a wannabe condition.

Then, ask them to finish their translation according to the use:

Exp	un_nx	un_ex	un_cx (t, f)
Ex(e)	sxp(e)	e	cjump($e \neq 0$, t, f)
Cx($a < b$)	seq(sxp(a), sxp(b))		
Nx(s)			

Prototranslation, Expression Shells

Rather, delay the translation until the use is known:

- **Ex** Expression shell, encapsulation of a proto value,
- **Nx** Statement shell, encapsulating a wannabe statement,
- **Cx** Condition shell, encapsulating a wannabe condition.

Then, ask them to finish their translation according to the use:

Exp	un_nx	un_ex	un_cx (t, f)
Ex(e)	sxp(e)	e	cjump($e \neq 0$, t, f)
Cx($a < b$)	seq(sxp(a), sxp(b))	eseq(t $\leftarrow$($a < b$), t)	
Nx(s)			

Prototranslation, Expression Shells

Rather, delay the translation until the use is known:

- **Ex** Expression shell, encapsulation of a proto value,
- **Nx** Statement shell, encapsulating a wannabe statement,
- **Cx** Condition shell, encapsulating a wannabe condition.

Then, ask them to finish their translation according to the use:

Exp	un_nx	un_ex	un_cx (t, f)
Ex(e)	sxp(e)	e	cjump(e $\neq$ 0, t, f)
Cx(a $<$ b)	seq(sxp(a), sxp(b))	eseq(t $\leftarrow$(a $<$ b), t)	cjump(a $<$ b, t, f)
Nx(s)			

Prototranslation, Expression Shells

Rather, delay the translation until the use is known:

- **Ex** Expression shell, encapsulation of a proto value,
- **Nx** Statement shell, encapsulating a wannabe statement,
- **Cx** Condition shell, encapsulating a wannabe condition.

Then, ask them to finish their translation according to the use:

Exp	un_nx	un_ex	un_cx (t, f)
Ex(e)	sxp(e)	e	cjump($e \neq 0$, t, f)
Cx($a < b$)	seq(sxp(a), sxp(b))	eseq(t $\leftarrow$($a < b$), t)	cjump($a < b$, t, f)
Nx(s)	s		

Prototranslation, Expression Shells

Rather, delay the translation until the use is known:

- Ex Expression shell, encapsulation of a proto value,
- Nx Statement shell, encapsulating a wannabe statement,
- Cx Condition shell, encapsulating a wannabe condition.

Then, ask them to finish their translation according to the use:

Exp	un_nx	un_ex	un_cx (t, f)
Ex(e)	sxp(e)	e	cjump($e \neq 0$, t, f)
Cx(a $<$ b)	seq(sxp(a), sxp(b))	eseq(t $\leftarrow$(a $<$ b), t)	cjump(a $<$ b, t, f)
Nx(s)	s	**error**	

Prototranslation, Expression Shells

Rather, delay the translation until the use is known:

- **Ex** Expression shell, encapsulation of a proto value,
- **Nx** Statement shell, encapsulating a wannabe statement,
- **Cx** Condition shell, encapsulating a wannabe condition.

Then, ask them to finish their translation according to the use:

Exp	un_nx	un_ex	un_cx (t, f)
Ex(e)	sxp(e)	e	cjump($e \neq 0$, t, f)
Cx(a $<$ b)	seq(sxp(a), sxp(b))	eseq(t $\leftarrow$(a $<$ b), t)	cjump(a $<$ b, t, f)
Nx(s)	s	**error**	**error**

if 11 < 22 | 22 < 33 then print_int(1) else print_int(0)

```
cjump ne
    eseq seq cjump lt const 11 const 22 name l0 name l1
                label l0  move temp t0 const 1
                          jump name l2
                label l1  move temp t0
                          eseq seq move temp t1 const 1
                                   cjump lt const 22 const 33 name l3 name l4
                                   label l4
                                   move temp t1 const 0
                                   label l3
                               seq end
                               temp t1
                          jump name l2
                label l2
            seq end
            temp t0
    const 0
    name l5
    name l6
label l5   sxp call name print_int const 1
           jump name l7
label l6   sxp call name print_int const 0
           jump name l7
label l7
```

A Better Translation: Ix

```
seq
    cjump lt const 11 const 22 name l3 name l4
    label l3
      cjump ne const 1 const 0 name l0 name l1
    label l4
      cjump lt const 22 const 33 name l0 name l1
  seq end
label l0
  sxp call name print_int const 1
  jump name l2
label l1
  sxp call name print_int const 0
label l2
```

Summary